INTEGRATIVE PROGRAMMING AND TECHNOLOGIES

Module 1 – Introduction to Integrative Programming

Overview

Modern software applications rarely rely on a single programming language or technology. Instead, developers combine multiple technologies such as programming languages, databases, APIs, cloud services, and version control systems to build complete, scalable, and maintainable applications. This process is known as **Integrative Programming**.

This module introduces the concepts of system integration, client-server architecture, APIs, technology stacks, software development methodologies, version control, system design, and database fundamentals.

Learning Outcomes

At the end of this lesson, students should be able to:

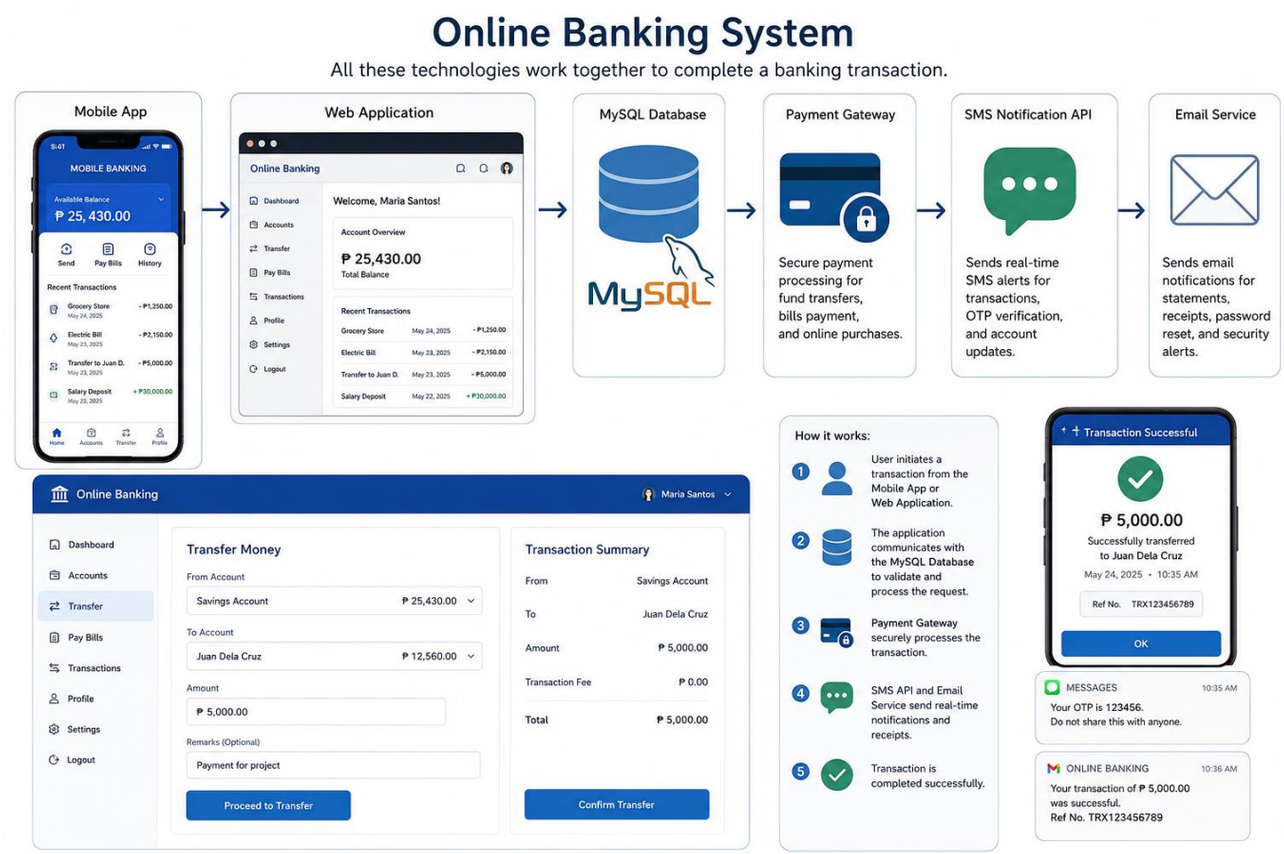
- Explain Integrative Programming and its importance.
- Describe how system integration works.
- Differentiate client and server roles.
- Explain the function of APIs and technology stacks.
- Compare different software development methodologies.
- Use Git and GitHub for version control.
- Interpret system design diagrams.
- Review relational databases and SQL concepts.

What is Integrative Programming?

Integrative Programming is the process of combining different software technologies, programming languages, databases, frameworks, and external services into a single, functional application.

Instead of developing isolated components, integrative programming ensures that different systems communicate and work together efficiently.

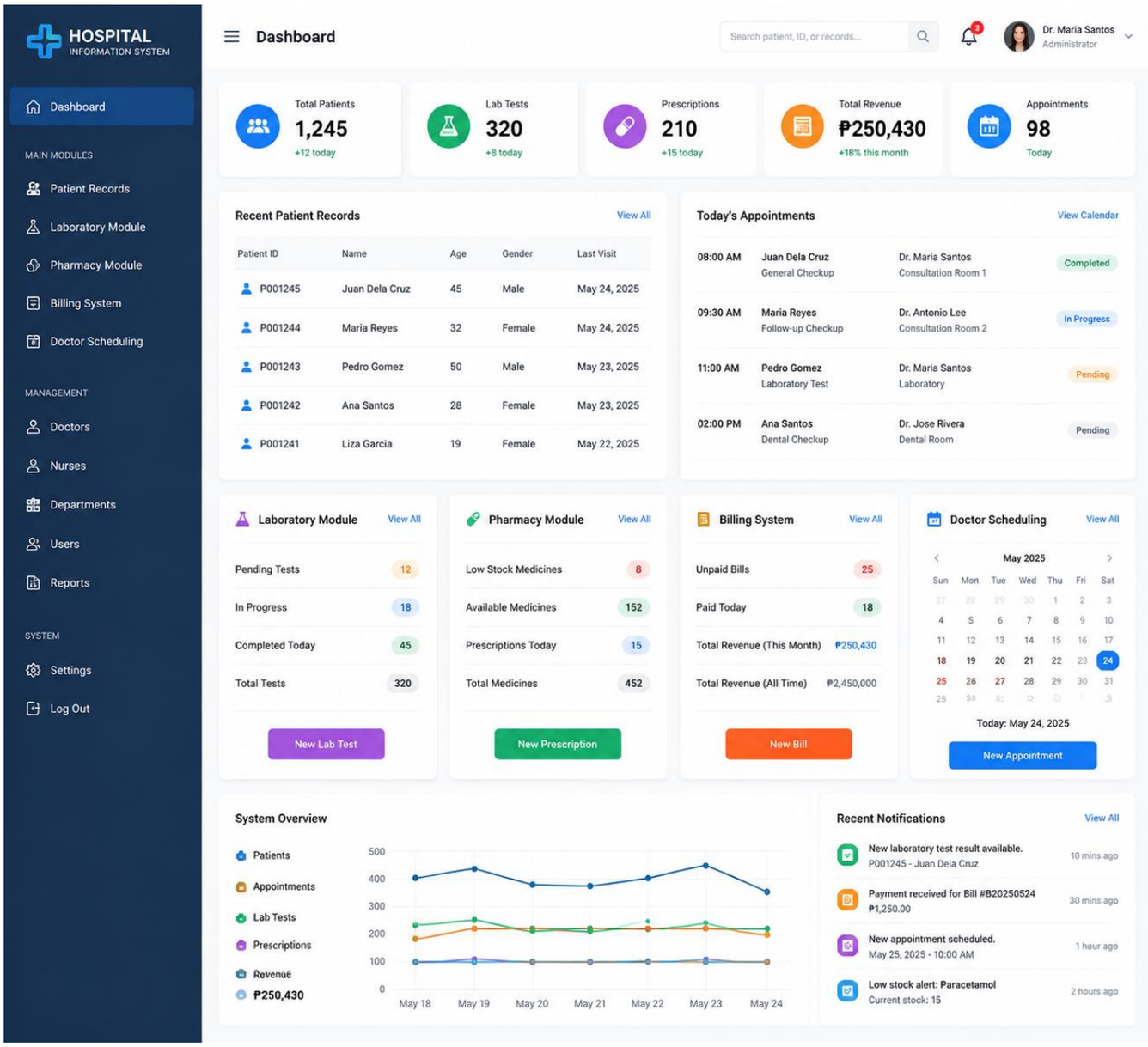
Examples:



Hospital Information System

A hospital system integrates:

- Patient Records
- Laboratory Module
- Pharmacy Module
- Billing System
- Doctor Scheduling



NOTE: Modern software systems are rarely built using a single technology. Developers must understand how different components interact to create efficient, scalable, and secure applications.

System Integration Concepts

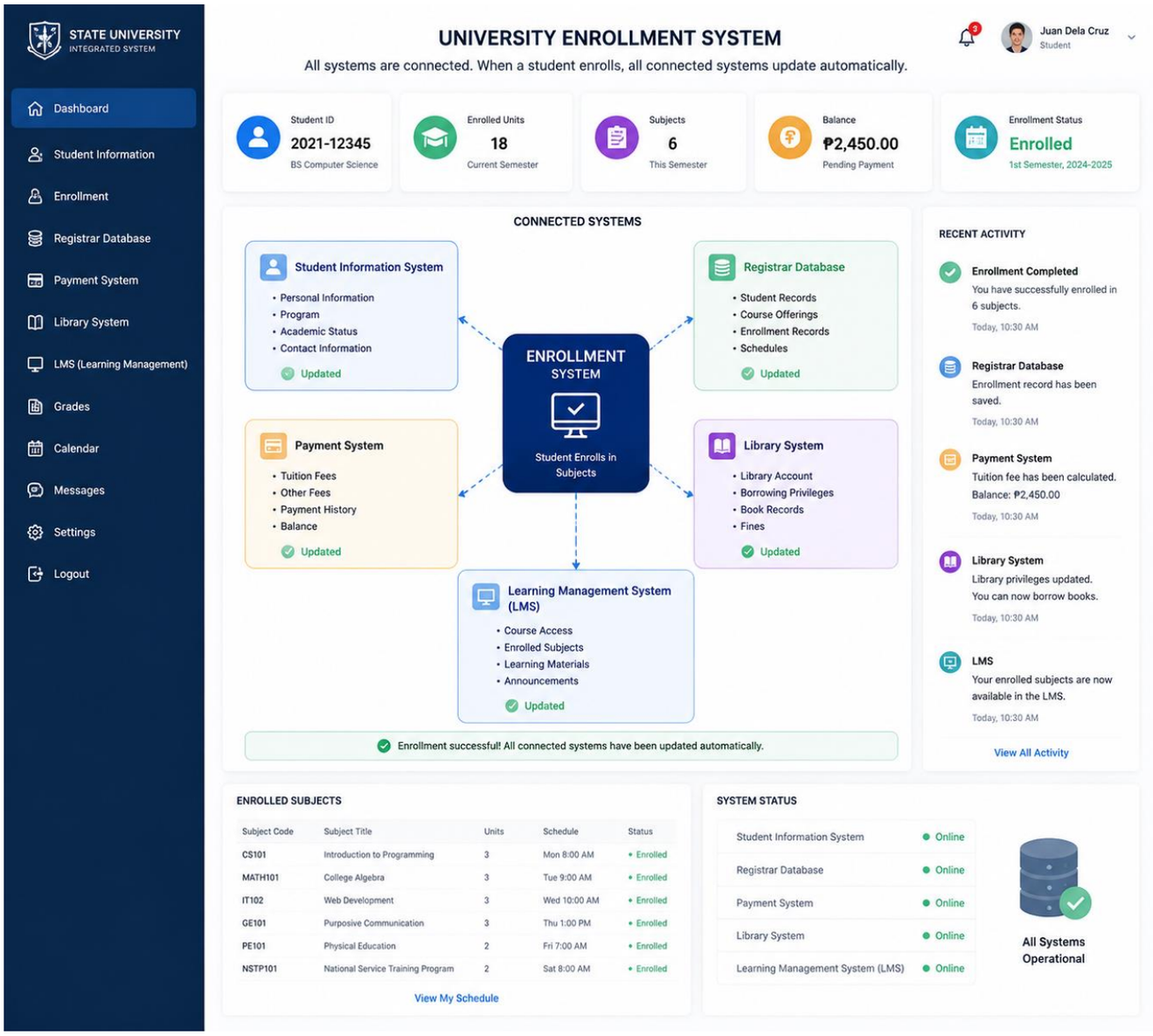
System Integration is the process of connecting different software systems, hardware devices, or applications so they can exchange information and work as one complete system.

Example

A university enrollment system connects:

- Student Information System
- Registrar Database
- Payment System
- Library System
- Learning Management System (LMS)

When a student enrolls, all connected systems update automatically.



NOTE: Without system integration, organizations would have isolated systems requiring duplicate data entry, increasing errors and reducing efficiency.

Client-Server Architecture

Client-server architecture is a computing model where the **client** requests services or data, and the **server** processes the request and returns the response.

Components of Client-Server Architecture

Client

- **Browser** – A software application used to access and interact with web-based systems.
  Example: Google Chrome accessing an online banking website.
- **Mobile App** – A smartphone application that allows users to access services through mobile devices.
  Example: GCash mobile application used for sending money and paying bills.
- **Desktop Application** – A software program installed on a computer that communicates with a server to perform tasks.
  Example: A hospital management system installed on a clinic computer.

Server

- **Database** – A storage system that manages and organizes application data.
  Example: MySQL database storing student records in a university system.
- **Application Server** – Processes business logic and handles requests between the client and database.
  Example: A banking application server verifying account balances before transferring money.

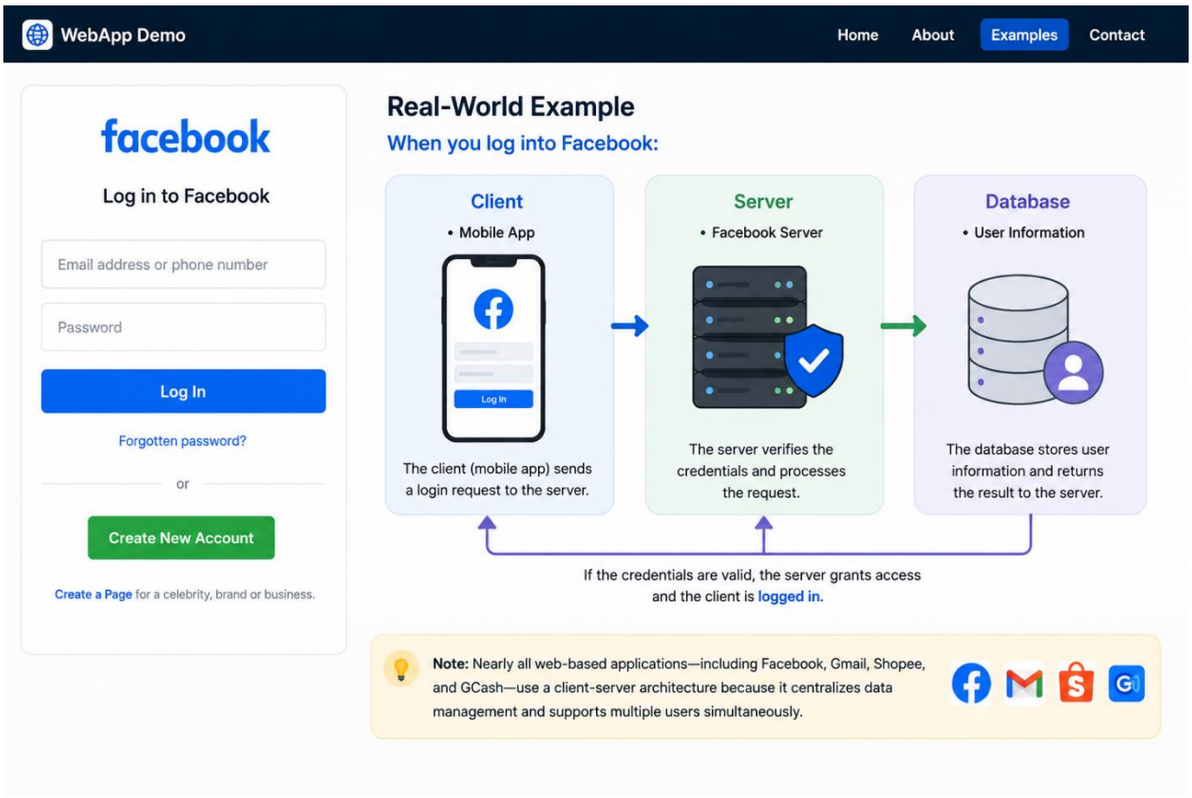
- **Web Server** – Delivers web pages and resources to users through internet requests. **Example:** Apache or Nginx serving an online shopping website.

Example

When you log into Facebook:

Client: Mobile App | **Server**: Facebook Server | **Database**: User Information

The client sends a login request, and the server verifies the credentials before granting access.



Note: Nearly all web-based applications—including Facebook, Gmail, Shopee, and GCash—use a client-server architecture because it centralizes data management and supports multiple users simultaneously.

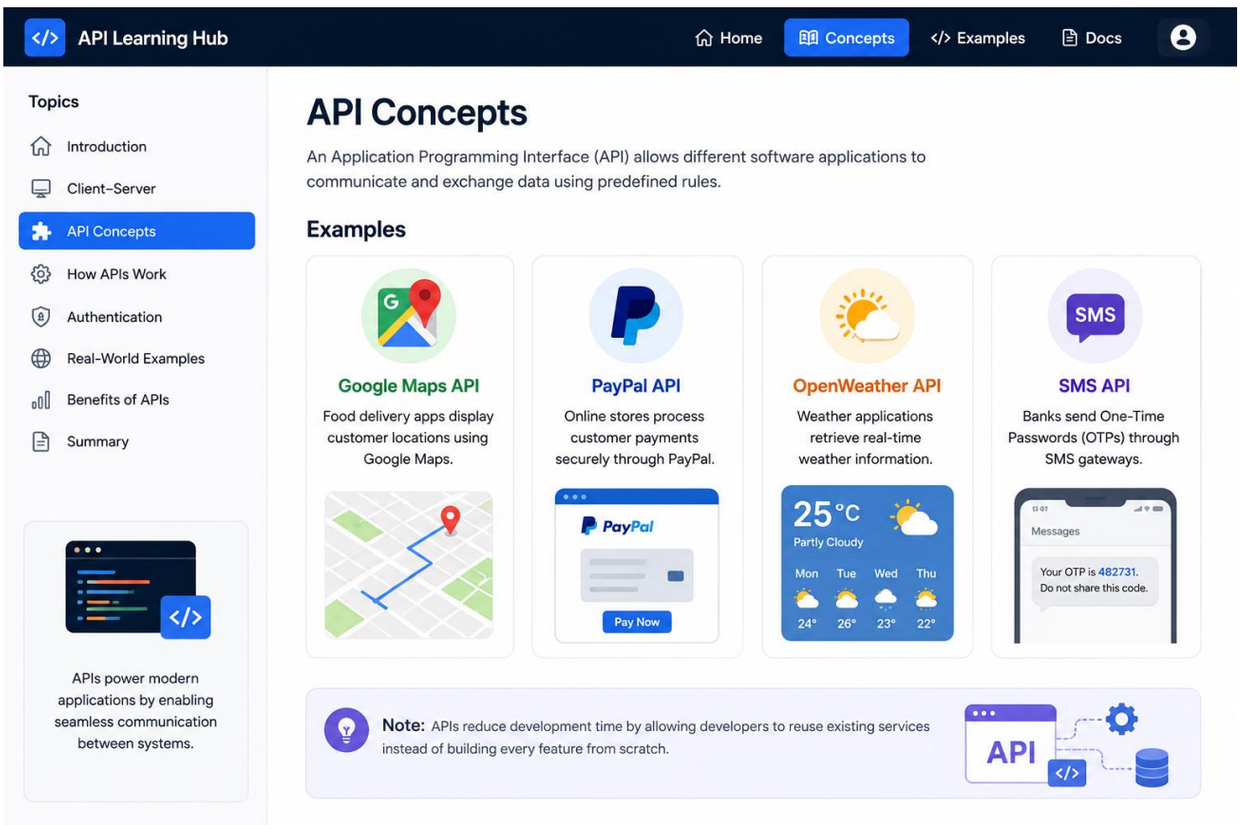
https://www.youtube.com/watch?v=frzVtaNrHU0. Facebook datacenters

A **data center** is a secure facility that houses servers, storage, and networking equipment used to store, process, and manage data for websites, applications, and online services.

API Concepts

An **Application Programming Interface (API)** allows different software applications to communicate and exchange data using predefined rules.

Example:



Technology Stacks

A **Technology Stack (Tech Stack)** is the collection of programming languages, frameworks, databases, and tools used to build an application.

Example Technology Stacks